

智慧银行实验教程

实验三：MCP 协议实践

姓名 学号 班级

2026 年 6 月 25 日

1 实验目的

1. 理解 MCP（Model Context Protocol，模型上下文协议）的架构原理与工作模式；
2. 掌握 MCP 服务器配置文件的格式与语法（JSON），能够独立编写和修改 `mcp.json`；
3. 完成至少 3 个 MCP 服务器的接入与验证，本实验实际完成 8 个；
4. 学习 MCP 服务器的故障排查方法，掌握版本兼容性、权限配置等常见问题的解决策略；
5. 了解 MCP 协议在金融业务场景（数据分析、文档处理、网页抓取、浏览器自动化）中的应用价值；
6. 实践 MCP + Git + CNB 的集成工作流，通过贪吃蛇项目体验 AI 工具链的协同运作；
7. 初步了解 BMAD 方法论（Build-Measure-Analyze-Decide），为第 9 章综合项目开发奠定方法论基础。

2 实验环境

表 1: 实验环境一览

类别	项目	版本/说明
操作系统	Windows 10/11 (x64)	—
AI IDE	WorkBuddy	MCP 集成
Python	托管版本	3.13.12
Node.js	托管版本	v22.22.2
uv / uvx	pip 安装	0.11.24
.NET SDK	E:\dotnet	10.0.301
Stata	本地安装	Stata18 MP
MCP 配置路径	%USERPROF IILE%\workbuddy\mcp.json	—

3 实验步骤

3.1 步骤 1：理解 MCP 协议架构

在学习 MCP 协议之前，首先了解其基本架构。MCP 是一种开放协议，标准化了 AI 应用与外部数据源、工具之间的连接方式。其核心架构分为三层：

- **客户端层 (Host)：** AI IDE（如 WorkBuddy），负责发起工具调用请求；
- **传输层 (Transport)：** 支持 stdio（标准输入输出）和 HTTP 两种通信方式；
- **服务器层 (Server)：** 具体的 MCP 服务进程，提供文件读写、网页抓取、数据分析等能力。

类比：MCP 协议相当于 AI 世界的“USB-C 接口”，标准化的连接方式使不同的 AI 工具能够接入各种外部资源。

3.2 步骤 2：设计 MCP 服务器配置方案

按照课本附录 B 的推荐配置，结合智慧银行课程需求，设计以下 8 个 MCP 服务器的配置方案：

表 2: MCP 服务器配置方案

服务名	启动方式	核心包	金融业务用途
fetch	uvx	mcp-server-fetch	获取金融资讯、年报网页
playwright	npx	@playwright/mcp@latest	自动化操作网银系统
chrome-devtools	npx	chrome-devtools-mcp@latest	调试 Web 金融应用
excel	npx	@negokaz/excel-mcp-server	处理金融表格数据
word	uvx	office-word-mcp-server	生成金融分析报告
ppt	uvx	office-powerpoint-mcp-server	制作金融演示文稿
WindowsMCP.Net	dotnet tool	WindowsMCP.Net	桌面自动化操作
stata-mcp	uvx	stata-mcp	计量经济学回归分析

3.3 步骤 3: 编写 mcp.json 配置文件

根据配置方案, 编写完整的 mcp.json 配置文件。注意 JSON 语法要求: Windows 路径中的反斜杠需写成 \\, 环境变量使用 env 字段配置。

配置文件全文如下:

```
{
  "mcpServers": {
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    },
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    },
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "chrome-devtools-mcp@latest"]
    },
    "excel": {
      "command": "npx",
      "args": ["--yes", "@negokaz/excel-mcp-server"],
      "env": {
        "EXCEL_MCP_PAGING_CELLS_LIMIT": "10000"
      }
    },
    "word": {
      "command": "uvx",
      "args": ["--from", "office-word-mcp-server", "word_mcp_server"]
    },
    "ppt": {
      "command": "uvx",
      "args": ["--from", "office-powerpoint-mcp-server", "ppt_mcp_server"]
    },
    "WindowsMCP.Net": {
      "command": "E:\\\\u667a\\u6167\\u94f6\\u884c\\u5b9e\\u9a8c\\u4f5c\\u4e1a\\\\dotnet-tools\\\\Windows-MCP.Net.exe",
```

```
"args": []
},
"stata-mcp": {
  "command": "uvx",
  "args": ["stata-mcp"],
  "env": {
    "STATA_PATH": "E:\\Stata18\\StataMP-64.exe",
    "MCP_STATA_LOGLEVEL": "INFO"
  }
}
}
```

配置文件保存至 C:\Users\用户名\.workbuddy\.mcp.json。

3.4 步骤 4：逐服务验证可用性

对配置的 8 个 MCP 服务器逐一进行命令行验证，确保包可以正常下载和启动。

表 3: MCP 服务验证记录

服务	验证命令	结果
fetch	uvx mcp-server-fetch --help	✓ 成功（包）
playwright	npx @playwright/mcp@latest --help	✓ 成功
chrome-devtools	npx -y chrome-devtools-mcp@latest --help	✓ 成功
excel	npx --yes @negokaz/excel-mcp-server --help	✓ 成功（式）
word	uvx --from office-word-mcp-server word_mcp_server --help	✓ 成功
ppt	uvx --from office-powerpoint-mcp-server ppt_mcp_server --help	✓ 成功
WindowsMCP.Net	E:\...\Windows-MCP.Net.exe	✓ 成功
stata-mcp	STATA_PATH=... uvx stata-mcp --help	✓ 成功

3.5 步骤 5：Stata MCP 环境专项配置

stata-mcp 是本课程数据分析模块的核心服务，需要额外的环境配置：

1. 确认 Stata 路径：本机 Stata18 安装在 E:\Stata18\StataMP-64.exe，而非默认的 C:\Program Files\Stata18；
2. 设置环境变量：在 mcp.json 的 env 字段中指定 STATA_PATH 和 MCP_STATA_LOGLEVEL；
3. 验证 Stata 可执行：通过 Test-Path 确认文件存在。

3.6 步骤 6：MCP + Git 集成实践——贪吃蛇项目全流程

MCP 协议的价值不仅在于单个服务的配置，更在于多工具链的协同运作。本步骤通过贪吃蛇游戏项目，实践了从代码生成到仓库管理的完整 MCP + Git 集成工作流：

- 1. **代码生成阶段：**通过 AI IDE（WorkBuddy）的对话界面，描述贪吃蛇游戏的功能需求，AI 生成完整的 HTML/CSS/JavaScript 单文件游戏代码；
- 2. **本地保存：**将生成的代码文件保存至本地项目目录 E:\智慧银行实验作业\index.html；
- 3. **Git 版本管理：**将文件复制到本地 CNB 仓库副本，执行 `git add` → `git commit` → `git push` 完成版本提交；
- 4. **CNB 云端托管：**代码推送至远程仓库 `sichuanagr202301729/zhihuifyinhang`，可在浏览器中查看提交历史和文件内容。

此工作流体现了课本附录 D 所述的 CNB 操作流程：克隆仓库 → 添加文件 → 提交 → 推送 → 云端验证，与 GitHub 工作流完全兼容。

3.7 步骤 7：BMAD 方法论初探

BMAD（Build-Measure-Analyze-Decide）是一种 AI 驱动的敏捷开发方法论，将在本课程第 9 章中系统学习。在本次 MCP 实验中，已初步体验到 BMAD 的迭代思想：

表 4: MCP 配置中的 BMAD 要素		
BMAD 阶段	MCP 实验对应	具体体现
Build（构建）	MCP 配置编写	编写 mcp.json、安装工具链
Measure（度量）	服务验证	逐条验证 8 个服务是否可用
Analyze（分析）	故障排查	分析 Token 权限、版本兼容性问题
Decide（决策）	方案优化	决定迁移缓存、调整安装路径等

BMAD 方法论的核心价值在于将模糊的开发需求转化为可度量的迭代过程。在后续第 9-10 章的 CRM 系统开发中，将完整应用 BMAD 框架——从 PRD 文档编写到架构设计、UX 设计、Epic 拆分、Sprint 规划、代码实现和测试验证的完整流程。

4 实验结果

4.1 MCP 服务配置总览

表 5: MCP 服务配置完成状态

编号	服务名	工具	状态
1	fetch	uvx	✓ 已配置
2	playwright	npx	✓ 已配置
3	chrome-devtools	npx	✓ 已配置
4	excel	npx	✓ 已配置
5	word	uvx	✓ 已配置
6	ppt	uvx	✓ 已配置
7	WindowsMCP.Net	dotnet tool	✓ 已配置
8	stata-mcp	uvx	✓ 已配置

完成度：8/8 服务全部配置成功，超过课本要求的“至少配置 3 个 MCP 并验证”。

4.2 技术栈分类

按启动方式分类：

- **uvx 方式（4 个）**：fetch、word、ppt、stata-mcp ——Python 生态工具，通过 uv 统一管理；
- **npx 方式（3 个）**：playwright、chrome-devtools、excel ——Node.js 生态工具，通过 npm 管理；
- **dotnet tool 方式（1 个）**：WindowsMCP.Net ——.NET 生态工具，通过 dotnet CLI 管理。

三种技术栈的组合使用，体现了金融科技实践中多语言、多生态融合的特点。

5 问题与解决

5.1 问题 1: CNB Token 权限不足导致推送失败

现象：使用 Token 克隆仓库后尝试 `git push`，报“Permission denied”错误。

原因分析：初始创建的 Token（令牌名“123”）仅具备 `repo-code:r`（读取）权限，缺少 `repo-code:rw`（读写）权限。

解决：在 CNB 平台上重新创建具备完整读写权限的 Token（令牌名“1234”），更新 Git 远程地址后成功推送。

启示：MCP 服务与外部平台的交互涉及权限管理，配置前需明确所需权限范围。

5.2 问题 2：WindowsMCP.Net 包名差异

现象：按课本提示执行 `dotnet tool install -g Windows-MCP.Net`（带横杠）报错包名不存在。

原因分析：NuGet 官方仓库中的包名为 `WindowsMCP.Net`（无横杠），与课本附录中使用的名称略有差异。

解决：在 NuGet 官网 (<https://www.nuget.org/packages/WindowsMCP.Net>) 确认正确包名后，使用 `dotnet tool install --global WindowsMCP.Net` 安装成功。

5.3 问题 3：mcp.json 修改后不生效

现象：修改配置文件后重启 IDE，新配置的 MCP 服务未出现在可用工具列表中。

原因分析：按照课本附录 B.2.3 的排查指引，WorkBuddy 需要完全退出（任务栏右键退出）再启动，仅关闭窗口不够。

解决：通过任务栏图标完全退出 WorkBuddy，重新启动后所有 MCP 服务正常加载。

5.4 问题 4：playwright 浏览器未安装

现象：playwright MCP 服务可启动，但执行浏览器操作时报浏览器未找到。

原因分析：@playwright/mcp@latest 包本身不包含浏览器二进制文件，需单独下载 Chromium/Firefox/WebKit（约 500 MB）。

解决：执行 `npx playwright install` 下载浏览器。因文件较大，建议在网络条件良好时进行。当前仅完成 MCP 配置，浏览器安装待后续实验时执行。

6 实验心得

通过本次 MCP 协议实践实验，我对“AI 连接金融世界”这一核心理念有了具体而深刻的理解：

理论认知提升：

1. MCP 协议的“客户端—传输—服务器”三层架构设计优雅而实用，标准化的接口使不同生态的工具能够统一接入 AI 助手；

2. 8 个 MCP 服务涵盖了金融业务的核心场景:数据获取(fetch)、系统操作(playwright/chrome-devtools/WindowsMCP.Net)、统计分析(stata-mcp)、办公文档(excel/word/ppt),构建了完整的金融 AI 工具链。

实践能力提升:

1. 掌握了 JSON 配置文件的编写规范,特别是 Windows 路径转义、环境变量嵌入等细节;
2. 学会了跨平台工具链的协同配置(uvx/npx/dotnet 三种方式),理解了不同生态的管理哲学;
3. 在多次排障过程中(权限错误、包名差异、版本兼容性),积累了 MCP 服务的故障诊断经验。

不足与改进:

1. 对 MCP 协议的服务端开发(编写自定义 MCP Server)尚缺乏实践,仅完成了客户端配置。后续可在第 9 章 BMAD 方法论的综合项目中尝试开发面向银行业务的定制 MCP 服务;
2. chrome-devtools MCP 的远程调试功能尚未深入测试,实际使用效果待验证;
3. 多个 MCP 服务并发运行时的资源占用情况未评估,实际使用时需注意性能管理。

AI 协作反思: MCP 协议的配置过程本身就是 AI 协作的典范——AI 助手帮助生成了完整的配置文件、自动检查了路径正确性、实时排错了各种问题。这种“AI 调配 AI 工具”的元层次体验,让我直观感受到第 9 章将要学习的 BMAD 方法论中的“Build-Measure-Analyze-Decide”迭代思想。

从贪吃蛇到 CRM 系统的能力跃迁:贪吃蛇游戏的开发虽然简单,但它是单文件 HTML 项目的极端案例——AI 能够一次性生成完整可运行的代码。而当项目复杂度提升到 CRM 系统级别时(多模块、前后端分离、数据库设计、认证授权),单文件模式就不可行了。这正是 BMAD 方法论的价值所在:通过 PRD 文档明确需求、架构设计确定模块边界、Epic 拆分控制复杂度、Sprint 迭代逐步交付。从“AI 生成代码”到“AI 辅助架构”,是从工具使用者到系统构建者的关键跃迁。